



C2

Steer it & prove it

~50 min · Part A before the break (prompting + evals) · Part B after (the build)

BY THE END OF THIS CAPSULE YOU WILL BE ABLE TO

Choose the right prompting technique for a given failure, force output into a shape a program can check, and prove a prompt got better with an eval set you wrote yourself.

You can't touch the weights

The model is frozen. Everything you control sits in the prompt — and in how you check the answer.

What you CANNOT change	What you CAN change
the model's parameters (weights)	the instruction — what you ask, and how precisely
what it learned in pre-training	the context — facts, documents, examples you supply
its tendency to sound confident	the format — force an answer a program can check
how it tokenises your text	the effort — how much reasoning you pay for

Fine-tuning does change the weights — but it is slower, costlier, and rarely the first move. Prompting is where you start.

Takeaway Prompting is programming in words — the prompt is the only part of the machine you own.

The prompting ladder

Try these in order. Each rung costs a little more — in effort, tokens, or both.

	Technique	Use it when...	Cost
1	Be clear & direct	the model guessed what you meant — say the task, audience, format, constraints	free
2	Few-shot (examples)	it understands but formats or labels inconsistently	a few tokens
3	Chain-of-thought	it fails at logic, maths or multi-step analysis	longer outputs
4	Structured output	you need a machine-usable, checkable answer	~free
5	System / role prompt	tone or expertise drifts across a conversation	~free
6	Self-consistency	the answer must be reliable and you can afford to pay	n× the cost

Takeaway Pick the technique to the failure — don't stack all six by reflex.

Rung 1 · Be clear and direct

The cheapest rung, and the one most often skipped. Say the task, the audience, the format and the constraints.

VAGUE — the model has to guess

“Summarise this complaint.”

→ three paragraphs of prose, no decision,
no format your system can use, tone
pitched at nobody in particular

CLEAR — nothing left to guess

“You are triaging a bank complaint for the
duty manager. In 2 sentences, summarise
the issue. Then output exactly one word:
ESCALATE or MONITOR.”

→ 2 sentences, then ESCALATE

Four things to state, every time: **TASK** what to do · **AUDIENCE** who reads it · **FORMAT** the shape of the answer · **CONSTRAINTS** what to avoid

Treat the model like a capable new colleague with no context beyond your words — who will never ask a clarifying question.

Takeaway Most “the model is bad” moments are a prompt that never said what good looks like.

Rung 2 · Show, don't just tell

A “shot” is one worked example. Zero-shot = none; few-shot = two or three — usually the highest-leverage single move.

ZERO-SHOT — inconsistent

Classify the sentiment of this support ticket:

“The transfer failed twice today.”

→ “This appears somewhat negative,
indicating customer frustration...”

(prose, no fixed label set)

FEW-SHOT — consistent

“App keeps crashing.” → negative

“Fast and easy transfer!” → positive

“Where is the branch?” → neutral

“The transfer failed twice today.” →

→ negative

(one word, from a fixed label set)

Examples are not only for labels. Use them to pin a date format, a house tone, a redaction rule, or the exact shape of a record — e.g. “3 December 1988 → 1988-12-03” · “S\$4.2k → 4200” · “Dear Sir, → Hi there,”

Takeaway Examples pin down the format you want far more reliably than describing it in prose.

Under the hood — designing the examples

FOR BUILDERS

How many shots, which ones, and in what order.

- **How many** — 2–5 is usually the sweet spot; returns flatten fast and every example costs tokens on every call
- **Cover the space** — include the edge case and the “none of the above” case — not five variations of the easy one
- **Order matters** — models are sensitive to example order and to the last example especially; keep a fixed, tested order
- **Format exactly** — your examples ARE the spec — if you want JSON, the examples must be flawless JSON
- **Watch the cost** — few-shot examples are re-sent on every single call — a Capsule-3 line item

Takeaway Examples are code — version them, test them, and keep them minimal.

Rung 3 · Let it think out loud

Chain-of-thought: ask for the reasoning steps before the answer.

The task

An applicant earns S\$4,200/month and already repays S\$1,800/month. Under a 55% total debt-servicing limit, what is the maximum NEW monthly repayment?

Prompt	What comes back
"...what is the maximum new repayment?"	"Approximately S\$2,310." ✗ (forgot the existing debt)
"...Think step by step, then give the figure."	" $55\% \times 4,200 = 2,310 \rightarrow 2,310 - 1,800 = \text{S\$}510$." ✓

Trade-off: more reasoning tokens = more cost and latency. Newer "reasoning" models do much of this internally.

Takeaway Chain-of-thought buys accuracy on multi-step problems — and you pay for it in tokens.

Which rung fixes this prompt?

“Summarise this customer complaint and tell me if we should escalate.” → returns 3 paragraphs of prose; your ticketing system needs a yes/no flag.

A · Few-shot examples

B · Chain-of-thought

C · Structured output (a schema)

D · A system prompt

Not covered yet — a system prompt is a standing instruction you set once (“You are a compliance analyst at a Singapore bank...”) that applies to every turn, instead of repeating it in each message. That is rung 5, two slides from now.

Diagnose the failure first, then choose the rung.

Rung 4 · Structured output — the build-critical one

JSON is just labelled data — named fields with values, instead of a paragraph. Force the answer into that shape and a program can consume it, a test can check it.

PROSE — human-readable only

The applicant, Tan Wei Ming, earns about 4,200 dollars a month and is asking for an 80k loan...

```
{ "full_name": "Tan Wei Ming",  
  "monthly_income": 4200,  
  "currency": "SGD",  
  "loan_amount": 80000 }
```

IN YOUR PROMPT — this is what forces it, at the input stage

“Return ONLY a JSON object — no prose, no code fences — exactly this shape:

```
{"full_name": string, "monthly_income": number, "currency": "SGD"|"USD"|"CNY", "loan_amount": number}
```

If a field is missing, use null. Never invent a value.”

- **Now a program can use it** — no parsing prose, no regex guessing — read the field

Now a test can check it — is the JSON valid? is every key present? is income a number? is the currency allowed?

This is what makes evals possible — you cannot automatically grade a paragraph; you can automatically grade a field

Takeaway Structure turns an answer into a component — and a component is something you can test.

Under the hood — making the shape guaranteed

FOR BUILDERS

Asking politely for JSON is not the same as enforcing it.

Approach	What it does	Reliability
“Reply in JSON” in the prompt	the model usually complies — until it adds a sentence before the brace	fragile
Few-shot JSON examples	demonstrates the exact shape you want	better
Tool use / JSON mode with a schema	the API constrains the output to your declared schema	strongest
Validate + retry in code	parse; on failure, re-prompt with the error message	the safety net

Always validate on your side anyway — treat model output like any untrusted input.

Takeaway Declare the schema, then verify it — belt and braces.

Rungs 5 & 6 · Set the role, then vote

Two smaller levers — one nearly free, one expensive.

Rung 5 · System / role prompt (cheap — but you pay it every turn)

“You are a compliance analyst at a Singapore bank. Use MAS terminology. If a required field is missing, return null — never guess.”

Sets tone, expertise and standing rules for the whole conversation, so you don’t repeat them every turn.

Not quite free: it is re-sent with every single request, so a long system prompt is billed on every turn. Keep it tight — and it is still worth it.

Rung 6 · Self-consistency (n× the cost)

Ask the same question n times (with some randomness), then take the majority answer.

Buys reliability on hard problems where a single sample is a coin-flip — and multiplies your token bill by n.

Med-PaLM 2 (your pre-read) stacked exactly this — and in Capsule 3 we count what it cost.

Takeaway Roles are cheap and underused; voting is powerful and expensive — spend it where being wrong is costly.

Five ways people break their own prompts

Most “the model is bad” complaints are one of these.

Anti-pattern	Why it fails	Do this instead
“Be creative, accurate, brief and thorough”	conflicting goals — the model splits the difference badly	rank the goals; name the one that wins
“Don’t hallucinate”	a negative instruction with no mechanism behind it	supply the source and say “answer only from this”
One mega-prompt doing five jobs	errors compound; you can’t tell which part failed	chain smaller prompts, one job each
No output format specified	you get prose your system can’t consume	declare a schema (rung 4)
Assuming it knows this week’s facts	the weights are frozen at training time	put the facts in the context (your Week-2 RAG)

Takeaway Before blaming the model, read your prompt as if you were the model.

So — did it actually get better?

You changed the prompt and the answer changed. That is not the same as improvement.

The trap

You tweak the prompt, look at one output, decide “yes, better,” and ship it.

You just tested one case, once, with the person who wrote the prompt as the judge.

Change three things at once and you cannot tell which one helped — or which one broke something else.

- **Vibes don’t scale** — one good-looking answer says nothing about the next thousand
- **You need a number** — a pass-rate over a fixed set of cases, so two prompt versions are comparable

Takeaway “Better, not just different” — and the only way to know is to measure.

Write the eval set first

Ten cases, fixed, written before you start tuning the prompt. This is your ruler.

What to include	How many	Why
Typical cases — the everyday input	~5	proves it works at all
Edge cases — missing field, odd format, foreign name	~3	this is where prompts quietly break
Adversarial / trick — contradictory or out-of-scope input	~2	proves it fails safely rather than confidently

Each case = an input + what a correct answer must satisfy

Not “the right answer” word-for-word — the CONDITIONS a right answer meets. e.g. no employer listed → must return "employer": null, not an invented company.

“Tuning” here means editing your prompt — not fine-tuning the model. The model is frozen; nothing about it changes in this capsule.

Takeaway The eval set is written once and frozen — otherwise you’re moving the goalposts as you tune.

Two layers of checking

Cheap deterministic checks first; expensive judgement only where you need it.

	What it checks	Example	Cost
L1 · assertions	deterministic, machine-checkable facts about the output	valid JSON · all keys present · income is a number · currency ∈ {SGD, USD, CNY}	~free, instant
L2 · judge	subjective quality a rule can't capture	is the summary faithful to the source? is the tone appropriate? is it safe?	tokens (LLM judge) or human time

Rule of thumb: push as much as you can into L1. Every check you can automate is one you will actually run.

Pass-rate

the % of your fixed eval cases that pass — one number you can compare between prompt versions.

Takeaway Automate the checkable; reserve judgement for what genuinely needs it.

Which of these is a good L1 assertion?

A · “The answer should be helpful”

B · “Output is valid JSON with keys: name, income, currency”

C · “The tone should feel professional”

D · “It should be accurate”

Checkable by a program, with no opinion involved?

The iterate loop

This is the whole method — and it is what you will run live after the break.



Why “read the failures, not the successes”

Passing cases tell you nothing you didn’t already know. The three that failed tell you exactly which rung to reach for.

And why temperature 0 — it makes the model deterministic: same prompt, same answer. So a change in pass-rate is your edit, not luck. Turn it up only when you want variety on purpose (rung 6).

Takeaway Write the ruler, change one thing, measure again. That loop is the job.

END OF PART A · 15-MIN BREAK

Next: we build it.

After the break — a structured-output prompt for loan-form extraction, then ten eval cases and a pass-rate we push upwards together.

What we're about to build

PART B · BUILD

One prompt, one schema, ten tests — and a number that goes up.

Step	What we do	You end up with
1	Take a messy loan-application form (synthetic data)	a realistic, imperfect input
2	Write a structured-output prompt with a strict JSON schema	a prompt that returns a component, not prose
3	Write 10 eval cases with L1 assertions	your ruler

Why this task

Field extraction is the most common real enterprise AI job — and it is checkable, so you can prove it works. Same pattern as KYC onboarding, invoice processing, claims intake and medical-form triage.

Takeaway A step up from Week 2: there you assembled a pipeline; here you steer a model and prove the result.

The task — messy form in, strict JSON out

The input is deliberately imperfect: a missing field, an odd date, a currency that isn't dollars.

```
LOAN APPLICATION (synthetic)
-----
Name : Tan Wei Ming
DOB  : 14 Mar 1994
Employer : (left blank)
Income : 4,200 per month
Requesting 80k over 5 years
Branch note: walk-in, Tues
```

```
{
  "full_name": "Tan Wei Ming",
  "date_of_birth": "1994-03-14",
  "employer": null,
  "monthly_income": 4200,
  "currency": "SGD",
  "loan_amount": 80000
}
```

The interesting part is the blank employer: a good prompt returns null; a bad one invents a company — confidently.

Takeaway The hard cases aren't the messy formats — they're the missing values.

The eval harness — ten cases, one number

Each case runs the prompt and checks assertions. The pass-rate is the score.

```
def check(out, case):  
    assert is_json(out)  
    assert keys(out) == FIELDS  
    assert isinstance(  
        out["monthly_income"], (int, float))  
    assert out["currency"] in ALLOWED  
    return True
```

Version	What changed	Pass
v1	plain instruction, no schema	0 / 10
v2	+ strict JSON schema	4 / 10
v3	+ 2 few-shot examples	10 / 10
v4	+ “missing → null”, ISO dates	9 / 10

One change per version — so every improvement is attributable to something you can explain.

Based on run with 10 eval set & openai/gpt-4o-mini

Takeaway Four prompt versions, one number each — that’s the difference between engineering and guessing.

Two notebooks — pick your lane

FOR BUILDERS

Same task, same harness. One needs no key or credits at all.

	Local (token-free)	API
Model	a small instruct model (Qwen2.5-1.5B-Instruct)	a small hosted chat model — the cheap tier, not the frontier one
Cost	free — runs in the Colab runtime	uses your key / credits — but pennies at this scale
Quality	weaker; you will see more failures	strong enough for this task; fewer, subtler failures
Why run it	see the mechanics, and that the METHOD is what matters	see production-grade behaviour

CHEAP, YET CLOSE ENOUGH TO FRONTIER — any of these runs the notebook fine

GPT-5.6 Luna \$0.10 / \$0.60 · **Gemini 2.5 Flash-Lite** \$0.10 / \$0.40 · **Claude Haiku 4.5** \$1 / \$5 per 1M tokens in / out

*Our notebook defaults to **openai/gpt-4o-mini** (\$0.15 / \$0.60). Prices move fast — check openrouter.ai/models. Aug 2026 figures.*

Both ship with a pre-run HTML export — read the whole run, every cell and output, without executing anything.

Takeaway The method transfers across models — a weaker model just fails more visibly.

Your turn — add one eval case, then re-run.

1 · In pairs: write ONE new eval case for a failure we haven't covered.

2 · Add its assertion to the harness and re-run all cases.

3 · Did the pass-rate hold at 10/10 — or did you break it?

Ideas: a name with no surname · income in RMB · two applicants on one form · a future-dated birth date.

8 minutes · we'll take three cases from the room.

What you just built — and where it goes

This is not a toy exercise; it is the spine of your assignment and the setup for the next capsule.

- **Assignment 1, part (iii)** — a structured-output prompt plus a 10-case eval harness and your pass-rate history — due end of Week 4
- **Your project** — the same loop applies to whatever problem you choose: define the output shape, write the ruler, iterate
- **Capsule 3, next** — every rung you just used has a price tag — few-shot examples on every call, chain-of-thought on every answer, self-consistency n times over. We put a real model's recipe on the meter.

Takeaway You can now steer a model and prove it improved — the last question is what that improvement costs.

CAPSULE 2 · TAKEAWAY

Prompting is programming in words.

Pick the rung to the failure, force a shape you can check, and let the pass-rate — not your gut — tell you it improved. Next: what all of that costs.

